Praktikum Modul 4 - Cookies, Session, dan Local Storage



Nama : Muhamad Haikal Dwi Arya
Hari/Tanggal : Sabtu, 10/10/2026
Kelas : D3 – 2A
NIM : 251511018

Politeknik Negeri Bandung
Jurusan Teknik Komputer dan Informatika

Source code: https://github.com/haikaldwiaryaa/Praktikum-Modul-4---Cookies-Session-dan-Local-Storage.git

Tugas 1 Login

A. Tangkapan Layar

1. **Isi table user di database setelah di hash**

db_tugas.users: 2 rows total (approximately)

id	username	password	nama_lengkap	created_at	updated_at
1	haikaltzy	$2y$12$h65BqgLGfVSxYJj.hO.PSO2u/oAsPxnyE...	muhamad haikal	2026-10-05 03:32:59	2026-10-05 03:32:59
2	cobacobatzy	$2y$12$r1Wq65ztwUsusQpclRzdkOS8gJozipkO...	cocacola	2026-10-05 03:32:59	2026-10-05 03:32:59

2. **Uji Coba Login Gagal**

Login

Masuk untuk membuka dashboard

[Username atau password salah.]

Username
haikaltzy

Password

Masuk

3. **Uji Coba Login Berhasil**

Dashboard [Logout]

Selamat datang, muhamad haikal!

Halaman ini hanya bisa dibuka setelah login.

Username: haikaltzy

4. **Halaman Dashboard**

Dashboard [Logout]

Selamat datang, muhamad haikal!

Halaman ini hanya bisa dibuka setelah login.

Username: haikaltzy

5. **Akses Dashboard di Jendela Incognito (Tanpa Login)**

127.0.0.1:8000/login

YouTube Maps Terjemahkan Facebook Instagram hub pencari

Login

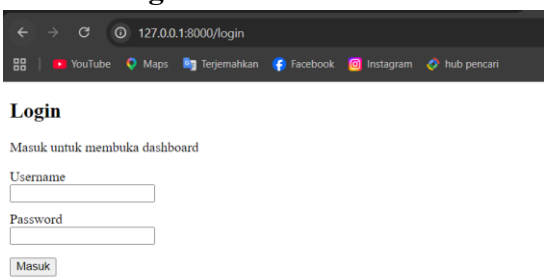
Masuk untuk membuka dashboard

Username

Password

Masuk

6. **Setelah Logout**



B. Jawaban Pertanyaan

1) Mengapa password harus di-hash? Lihat kolom password di tabel users dan jelaskan isinya.

   Password harus di-hash agar tidak tersimpan dalam bentuk teks biasa (*plain text*). Jika suatu saat database bocor atau diretas, data password asli pengguna tetap aman dan tidak bisa dibaca langsung oleh penyerang karena fungsi hash bersifat satu arah

   Isi kolom password pada tabel users adalah deretan string acak panjang yang diawali dengan $2y$12$.... String ini merupakan hasil enkripsi algoritma Bcrypt bawaan Laravel, yang di dalamnya sudah memuat informasi algoritma, tingkat keamanan,salt acak, dan nilai hash dari password tersebut.

2) Mengapa logout memakai metode POST dengan @csrf, bukan link biasa?

   Jika logout hanya menggunakan link biasa (metode GET), aplikasi akan rentan terhadap serangan CSRF (Cross-Site Request Forgery). Penyerang bisa menyisipkan tag tersembunyi seperti <img src="http://website.com/logout"> di situs lain, sehingga akun pengguna bisa tiba-tiba logout tanpa disadari saat membuka situs tersebut.

   Dengan menggunakan metode POST yang dilengkapi token @csrf, Laravel memastikan bahwa perintah logout benar-benar datang atas kehendak pengguna dari aplikasi itu sendiri, bukan manipulasi dari luar.

3) Apa fungsi middleware auth dan guest? Apa yang terjadi jika /dashboard dibuka tanpa login?

   - **Middleware auth**: Berfungsi sebagai penyaring akses, memastikan hanya pengguna yang sudah berhasil login yang dapat membuka halaman tertentu.
   - **Middleware guest**: Berfungsi sebaliknya, hanya mengizinkan pengguna yang belum login untuk mengakses halaman misalnya halaman /login. Jika yang membuka sudah login, mereka akan otomatis dialihkan ke dashboard.
   - **Jika /dashboard dibuka tanpa login**: Middleware auth akan langsung memblokir akses tersebut dan secara otomatis mengalihkan pengguna kembali ke halaman /login.

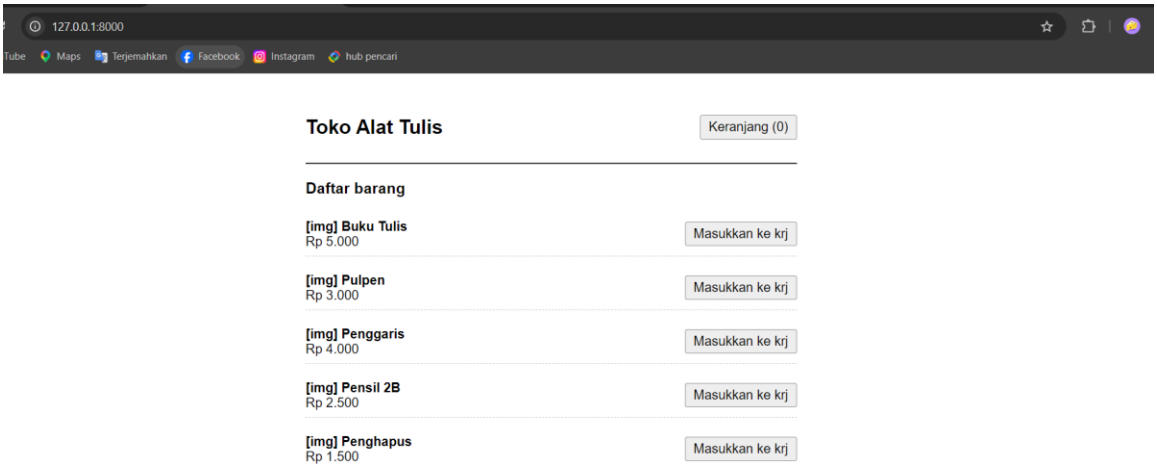
4) Apa fungsi session()->regenerate() setelah login berhasil?

   Fungsinya adalah untuk mengganti ID sesi (*session ID*) lama dengan yang baru saat status pengguna berubah dari pengunjung biasa menjadi pengguna login. Hal ini sangat penting untuk mencegah serangan Session Fixation, yaitu kondisi di mana penyerang sengaja menjebak korban dengan session ID tertentu sebelum login untuk membajak akun korban setelahnya.

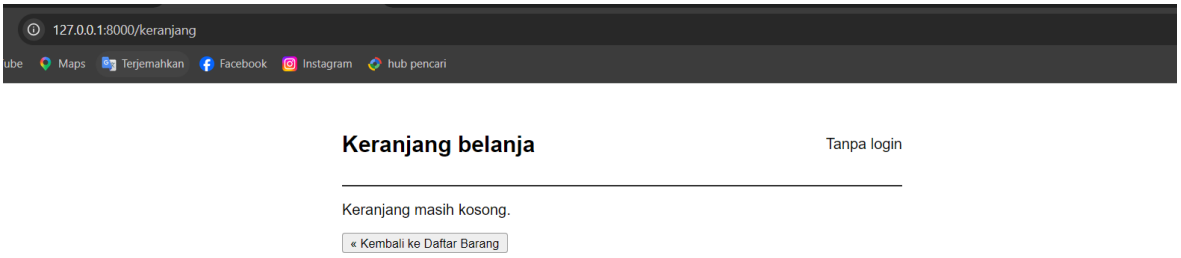
Tugas 2 keranjang belanja laravel

A. Tangkapan Layar

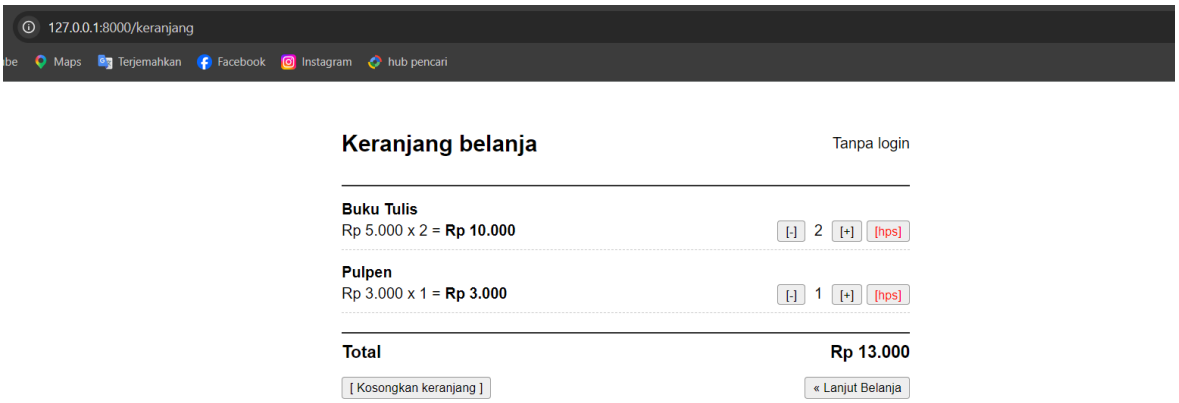
1. Halaman Daftar Barang



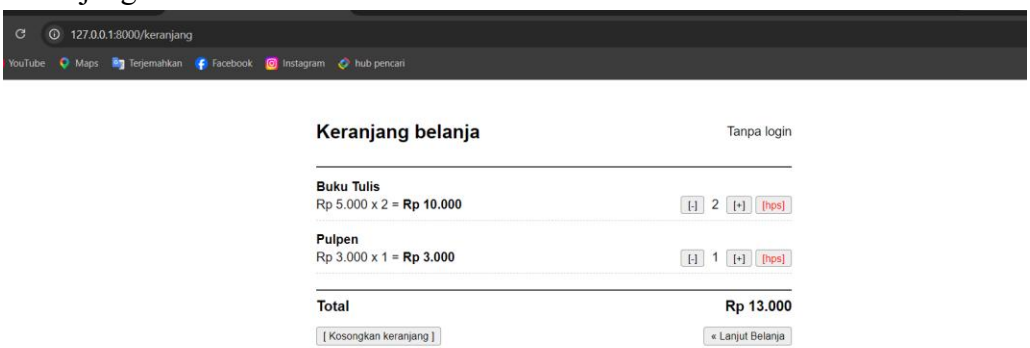
2. Keranjang Kosong



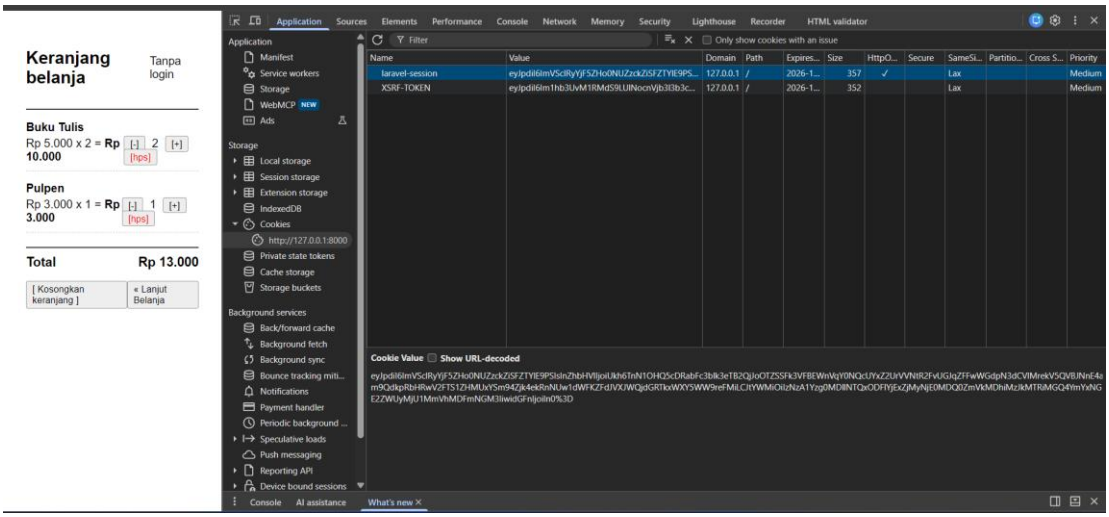
3. Keranjang Terisi



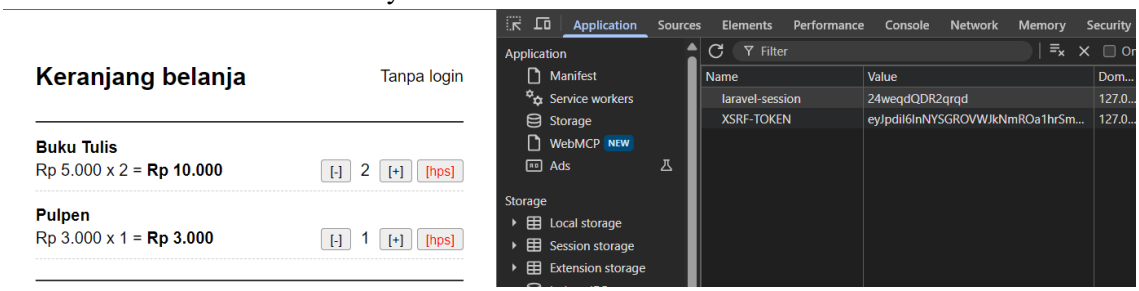
4. Keranjang Setelah di-Refresh



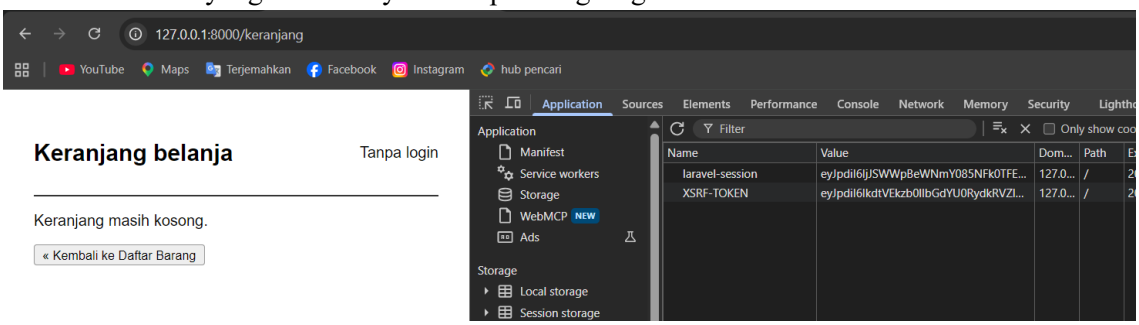
5. Lokasi Data di DevTools (Application > Cookies)



6. Percobaan Manipulasi di DevTools
Coba set random value session nya



Saat refresh data yang sebelumnya tersimpan langsung ke reset



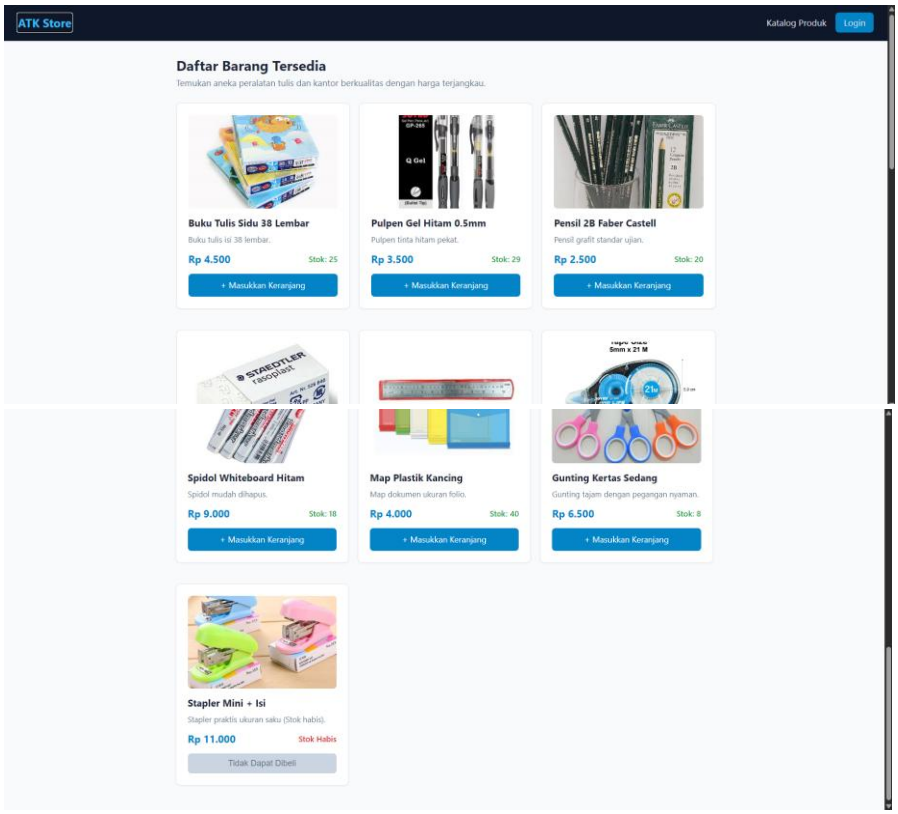
B. Jawaban Pertanyaan

1. **Mekanisme apa yang Anda pilih dan mengapa?** Saya memilih Session. Alasannya karena session tersimpan aman di server sehingga datanya tidak bisa dimanipulasi langsung oleh pengguna lewat browser, tapi pengunjung tetap bisa bertransaksi tanpa perlu login.
2. **Sebutkan satu kelebihan dan satu kekurangan pilihan Anda dibanding dua mekanisme lainnya:**
    - **Kelebihan:** Keamanannya tinggi, pengguna tidak bisa sembarangan mengedit data keranjang lewat DevTools seperti halnya pada cookie atau local storage.
    - **Kekurangan:** Membebani memori server karena server harus menyimpan file session untuk setiap pengunjung yang datang.
3. **Ubah data keranjang secara manual lewat DevTools (misal 999). Apa yang terjadi? Jika tidak bisa, jelaskan alasannya.** Tidak bisa diubah menjadi 999. Karena pada Laravel Session, browser hanya memegang cookie laravel_session yang terenkripsi acak, bukan data jumlah barangnya. Jika nilai cookie tersebut diubah manual di DevTools, Laravel akan menganggap sesinya rusak dan isi keranjang malah otomatis reset jadi kosong.
4. **Mengapa harga tidak boleh dipercaya dari data yang dikirim atau disimpan di browser? Apa yang harus dilakukan server saat checkout?** Karena data di browser mudah diubah siapa saja lewat inspect element. Kalau harga diambil dari browser, pengguna nakal bisa mengubah harga barang jadi Rp 1 sebelum bayar. Saat checkout, server wajib mengambil harga resmi langsung dari database berdasarkan ID barang, bukan dari data kiriman browser.
5. **Apa yang terjadi pada keranjang jika pengguna membuka situs dari browser lain?** Keranjang akan kosong. Karena cookie ID sesi tersimpan terpisah di masing-masing browser, jadi browser yang berbeda tidak saling berbagi sesi.

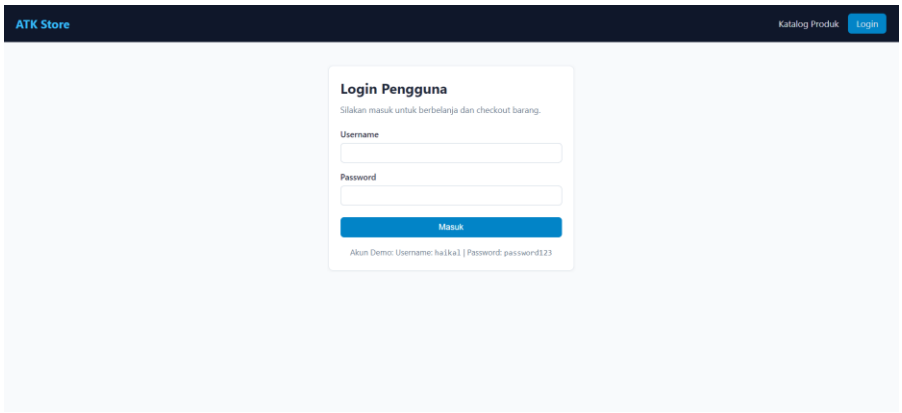
Tugas 3 Toko Online

A. Screenshoot Tampilan

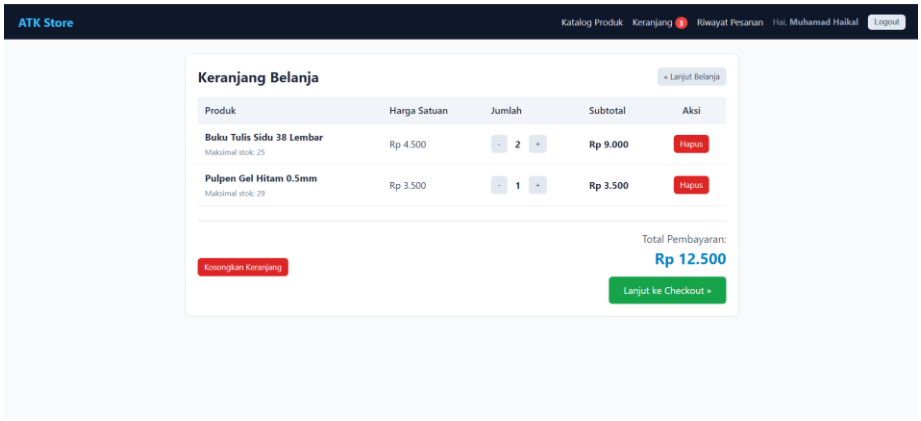
1. Katalog Produk



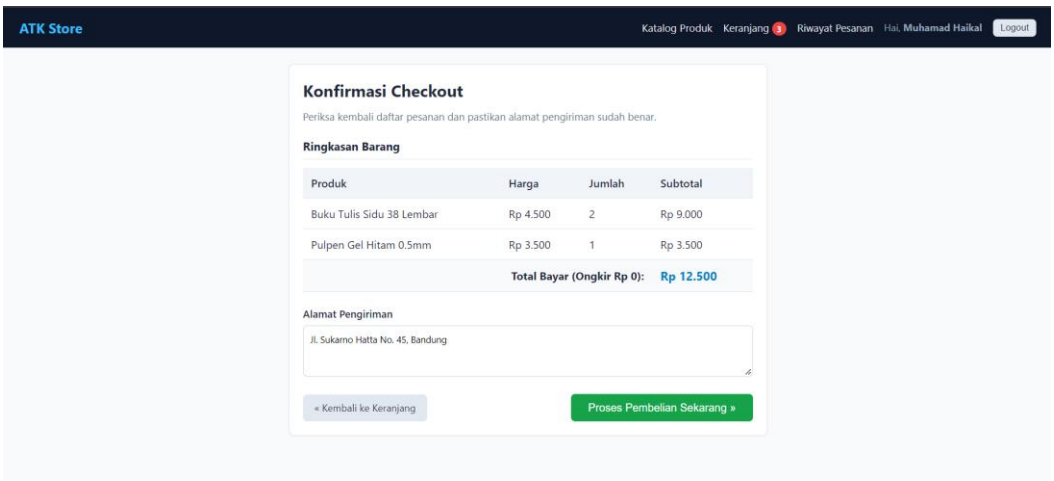
2. Login Page



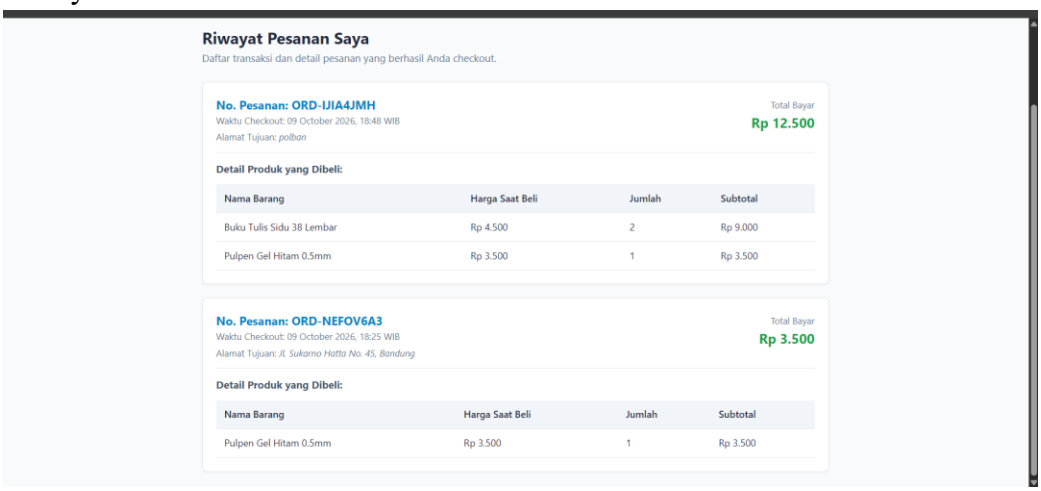
3. Keranjang



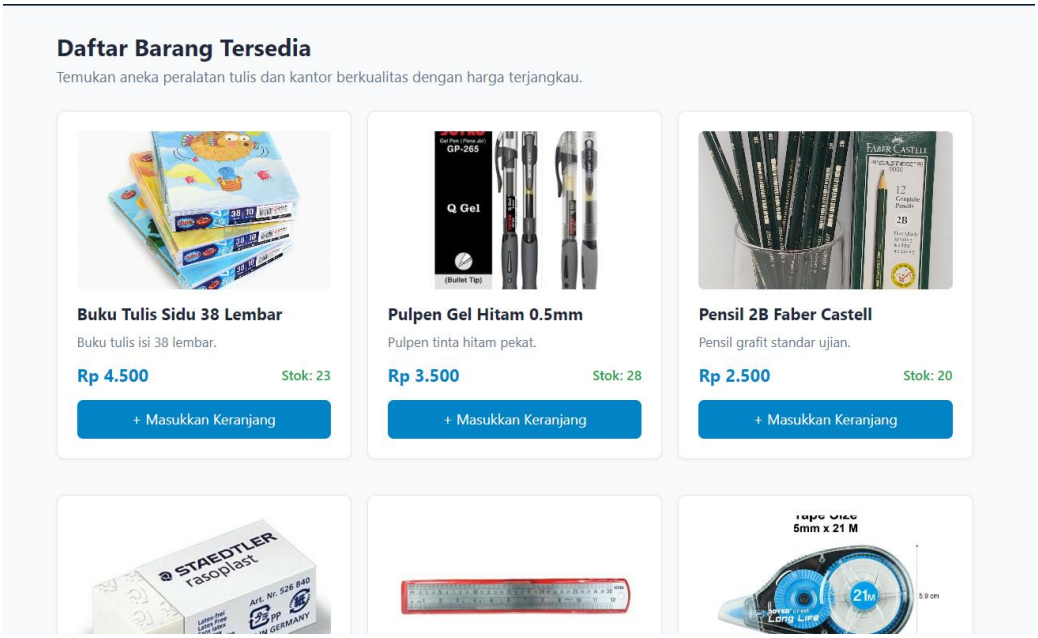
4. Halaman Checkout



5. Riwayat Pembelian



6. Update Stok Berkurang



C. Penjelasan Singkat Alur Program

1. **Akses & Wajib Login:**
    - Tamu bisa bebas lihat-lihat 10 barang di katalog tanpa login.
    - Tapi kalau klik tombol Masukkan Keranjang atau mau Checkout, otomatis dialihkan ke halaman Login dulu (diproteksi middleware auth).
2. **Validasi Stok Barang:**
    - Barang yang stoknya 0 (seperti Stapler Mini) tombolnya langsung mati dan bertuliskan "Tidak Dapat Dibeli".
    - Di keranjang, jumlah barang yang mau dibeli juga dibatasi biar nggak bisa melebihi stok yang ada di gudang.
3. **Keamanan Keranjang (Session):**
    - Keranjang disimpan di Session server, dan cuma nyimpen ID barang + jumlah beli.
    - Harga selalu ditarik langsung dari database, jadi aman dan nggak bisa diakalin lewat inspect element browser.
4. **Proses Checkout & Stok Berkurang:**
    - Pas klik checkout, sistem otomatis:
        1. Bikin nota pesanan baru (orders) dan simpan rincian barangnya (order_details).
        2. Motong sisa stok barang di database sesuai jumlah yang dibeli.
        3. Mengosongkan keranjang belanja.
    - Pakai DB::transaction, jadi kalau stok tiba-tiba nggak cukup, transaksi langsung dibatalkan biar data nggak kacau.
5. **Riwayat Pesanan:**
    - Pengguna bisa ngecek daftar barang apa aja yang pernah dibeli, lengkap sama tanggal order, total bayar, dan alamat kirimnya.

D. Refleksi Singkat

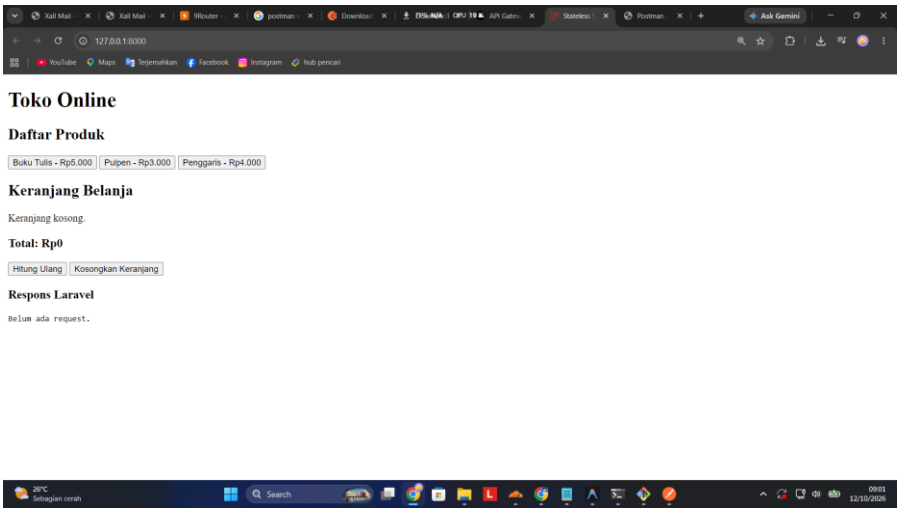
Inti dari tugas modul ini adalah paham kapan harus pakai tempat penyimpanan:

- Local Storage / Cookies: Bagus buat nyimpen data tampilan atau preferensi ringan di browser (kayak tema).
- Session: Wajib dipakai buat data penting kayak login dan keranjang belanja biar aman di sisi server dan nggak bisa dimanipulasi user.

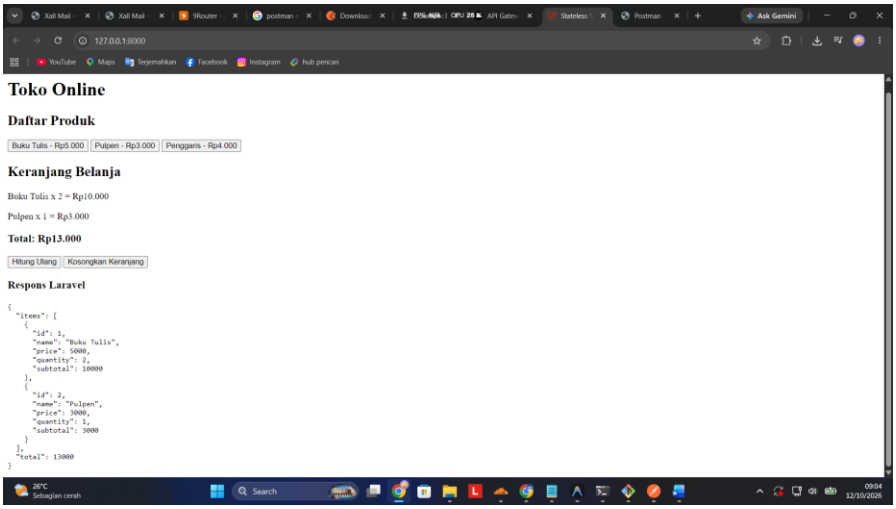
TUGAS 4B STATELESS

A. Screenshoot hasil percobaan yang dilakukan

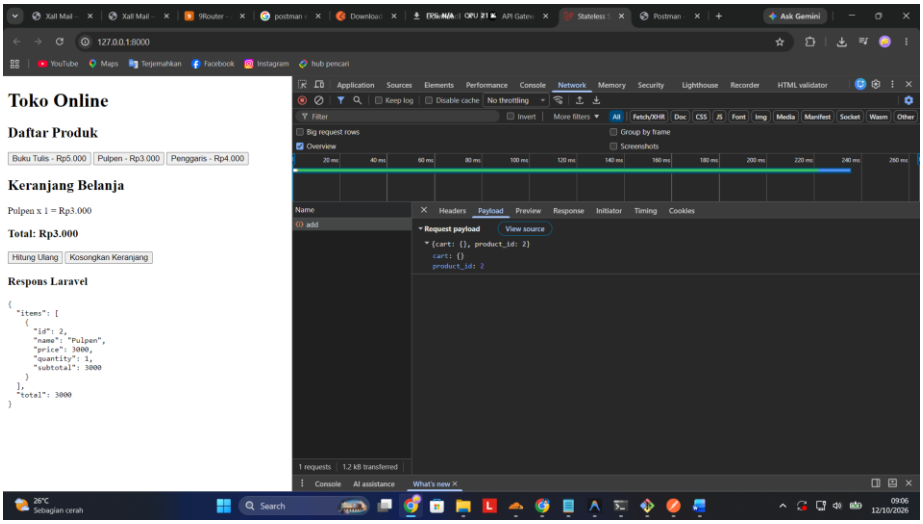
1. Menunjukkan tampilan web awal sebelum ada request



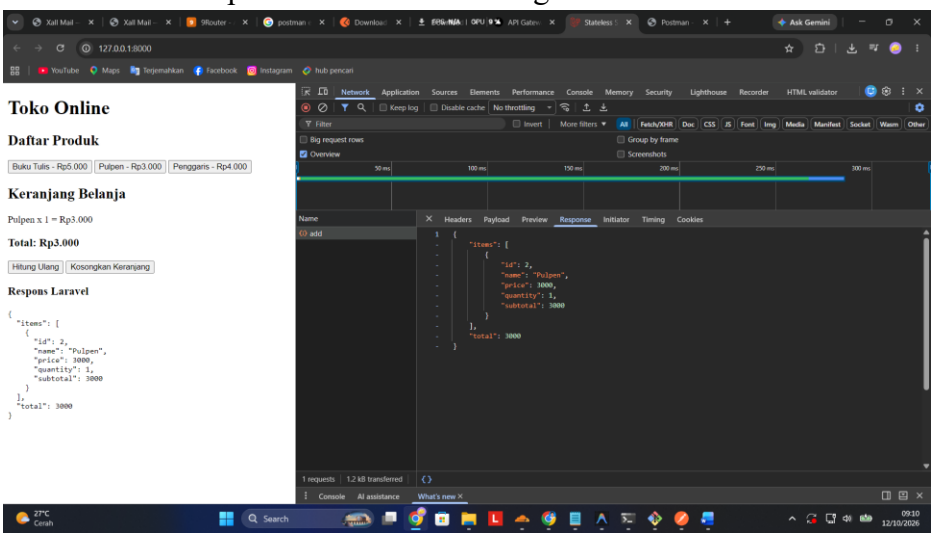
2. perhitungan kuantitas dan harga dari server Laravel



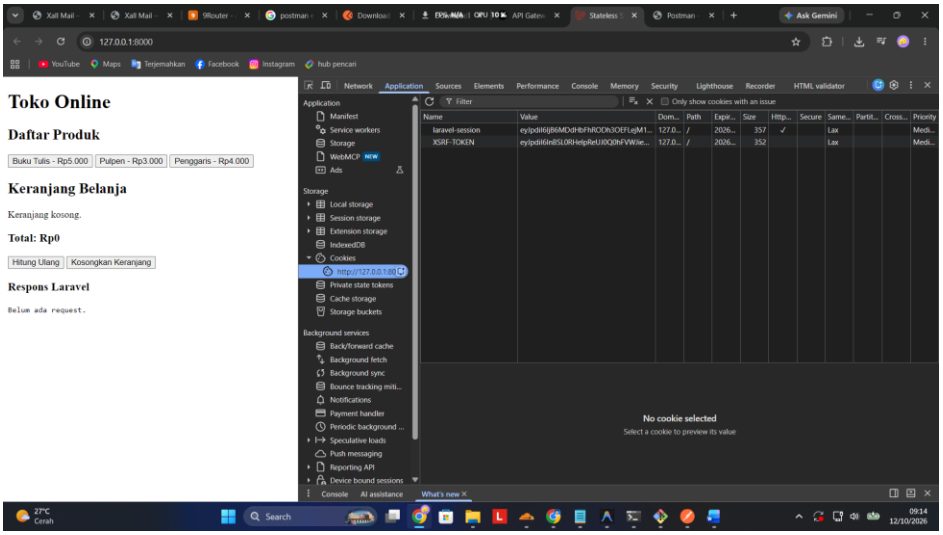
3. Membuktikan bahwa browserlah yang membawa data keranjang ke server (Stateless).



4. Membuktikan respon balik kalkulator harga dari Laravel dalam format JSON.



5. Membuktikan bahwa data keranjang tidak disimpan ke session maupun cookie (begitu di-refresh, keranjang reset).



B. Jawaban Diskusi dan Evaluasi

1. **Di baris kode mana browser mengirim state keranjang ke Laravel?**
   Di file shop.blade.php, tepatnya di dalam fungsi JavaScript post() pada baris body: JSON.stringify(payload). Payload ini membawa data variabel cart saat fungsi add() atau recalculate() dijalankan.
2. **Mengapa controller tidak perlu membaca session untuk menghitung keranjang?**
   Karena browser sudah mengirimkan seluruh daftar keranjang belanjaan lamanya langsung di dalam body request HTTP setiap kali tombol diklik. Jadi controller tinggal menghitung data yang masuk saat itu saja.
3. **Apa yang terjadi jika request dikirim ulang dengan payload yang sama?**
   Hasil responsnya akan selalu sama persis. Karena sifatnya stateless, server hanya menghitung data yang diterima saat itu tanpa ada efek samping perubahan data di server.

4. **Mengapa keranjang kosong setelah halaman di-refresh?**
   Karena data keranjang cuma disimpan sementara di memori variabel JavaScript (let cart = {}). Begitu halaman di-refresh (F5), halaman dimuat ulang dari awal sehingga variabelnya otomatis kereset jadi kosong, dan server memang tidak pernah menyimpannya ke session atau database.
5. **Apakah penggunaan middleware session Laravel berarti keranjang ini disimpan dalam session? Jelaskan.**
   Tidak. Middleware session di web.php aktif hanya untuk kebutuhan sistem bawaan Laravel seperti token CSRF. tetapi di dalam CartController sama sekali tidak ada kode yang menyimpan data keranjang ke session (session()->put()).
6. **Apa risiko keamanan jika server mempercayai harga yang dikirim browser?**
   Sangat berbahaya karena data di browser mudah diotak-atik lewat inspect element atau Postman. Pengguna nakal bisa saja mengubah harga barang dari Rp 5.000 menjadi Rp 1 sebelum dikirim. Makanya server wajib selalu mengambil harga asli dari sistem sendiri, bukan dari browser.
7. **Lesson-Learned (Stateful vs Stateless)**

C. Dari praktikum, saya jadi paham perbedaan nyata antara arsitektur stateful dan stateless:

- **Stateful (seperti Tugas 2 sebelumnya):**
  Server harus menyimpan catatan sesi pengguna. Kelebihannya, data keranjang aman dan tidak hilang saat halaman di-refresh. Tapi kekurangannya, makin banyak pengunjung yang datang, memori server akan makin terbebani untuk menyimpan sesi mereka.

- **Stateless (seperti Modul 4 Bagian 2 ini):**
  Server bekerja sangat ringan karena tidak perlu mengingat siapa pun dan cuma bertindak sebagai kalkulator pintar. Tanggung jawab menyimpan data ada di sisi klien (browser). Konsep stateless ini yang umum dipakai pada arsitektur modern seperti RESTful API dan aplikasi microservices karena server jadi jauh lebih mudah di-scale.